

BASICS

FUNCTIONAL PROGRAMMING IN ROCQ

Introduction

The *functional style* of programming is founded on simple, everyday mathematical intuitions: If a procedure or method has no side effects, then (ignoring efficiency) all we need to understand about it is how it maps inputs to outputs -- that is, we can think of it as just a concrete method for computing a mathematical function. This is one sense of the word "functional" in "functional programming." The direct connection between programs and simple mathematical objects supports both formal correctness proofs and sound informal reasoning about program behavior.

The other sense in which functional programming is "functional" is that it emphasizes the use of functions as *first-class* values -- i.e., values that can be passed as arguments to other functions, returned as results, included in data structures, etc. The recognition that functions can be treated as data gives rise to a host of useful and powerful programming idioms.

Other common features of functional languages include *algebraic data types* and *pattern matching*, which make it easy to construct and manipulate rich data structures, and *polymorphic type systems* supporting abstraction and code reuse. Rocq offers all of these features.

The first half of this chapter introduces some key elements of Rocq's native functional programming language, *Gallina*. The second half introduces some basic *tactics* that can be used to prove properties of Gallina programs.

Homework Submission Guidelines

If you are using *Software Foundations* in a course, your instructor may use automatic scripts to help grade your homework assignments. In order for these scripts to work correctly (and ensure that you get full credit for your work!), please be careful to follow these rules:

- Do not change the names of exercises. Otherwise the grading scripts will be unable to find your solution.
- Do not delete exercises. If you skip an exercise (e.g., because it is marked "optional," or because you can't solve it), it is OK to leave a partial proof in your `.v` file; in this case, please make sure it ends with the keyword `Admitted` (not, for example, `Abort`).
- It is fine to use additional definitions (of helper functions, useful lemmas, etc.) in your solutions. You can put these before the theorem you are asked to prove.
- If you introduce a helper lemma that you end up being unable to prove, hence end it with `Admitted`, then make sure to also end the main theorem in which you use it with

Admitted, not Qed. This will make sure you get partial credit if you use that main theorem to solve a later exercise.

You will also notice that each chapter (e.g., `Basics.v`) is accompanied by a *test script* (e.g., `BasicsTest.v`) that automatically calculates points for the finished homework problems in the chapter. These scripts are mostly for the auto-grading tools, but you may also want to use them to double-check that your file is well formatted before handing it in. In a terminal window, either type `"make BasicsTest.vo"` or do the following:

```
rocq compile -Q . LF Basics.v
rocq compile -Q . LF BasicsTest.v
```

See the end of this chapter for more information about how to interpret the output of test scripts.

There is no need to hand in `BasicsTest.v` itself (or `Preface.v`).

If your class is using the Canvas system to hand in assignments...

- If you submit multiple versions of the assignment, you may notice that they are given different names. This is fine: The most recent submission is the one that will be graded.
- If you are handing in multiple files at the same time (i.e., if more than one chapter is assigned in the same week), you should make a *single* submission with all the files at once by using the "Add another file" button just above the comment box.

Data and Functions

Enumerated Types

One notable thing about Rocq is that its set of built-in features is *extremely* small. For example, instead of the usual palette of atomic data types (booleans, integers, strings, etc.), Rocq offers a powerful mechanism for defining new data types from scratch, with all these familiar types as instances.

Naturally, the Rocq distribution also comes with an extensive standard library providing definitions of booleans, numbers, and many common data structures like lists and hash tables. But there is nothing magic or primitive about these library definitions. To illustrate this, in this course we will explicitly recapitulate almost all the definitions we need, rather than getting them from the standard library.

Days of the Week

To see how the datatype definition mechanism works, let's start with a very simple example. The following declaration tells Rocq that we are defining a set of data values -- a *type*.

```
Inductive day : Type :=
| monday
| tuesday
| wednesday
| thursday
| friday
| saturday
| sunday.
```

The new type is called `day`, and its members are `monday`, `tuesday`, etc.

Having defined `day`, we can write functions that operate on days.

```
Definition next_working_day (d:day) : day :=
match d with
| monday => tuesday
| tuesday => wednesday
| wednesday => thursday
| thursday => friday
| friday => monday
```

```
| saturday => monday
| sunday => monday
end.
```

Note that the argument and return types of this function are explicitly declared on the first line. Like most functional programming languages, Rocq can often figure out these types for itself when they are not given explicitly -- i.e., it can do *type inference* -- but we'll generally include them to make reading easier.

Having defined a function, we can check that it works on some examples. There are actually three different ways to do examples in Rocq. First, we can use the command `Compute` to evaluate a compound expression involving `next_working_day`.

```
Compute (next_working_day friday).
(* ==> monday : day *)

Compute (next_working_day (next_working_day saturday)).
(* ==> tuesday : day *)
```

(We show Rocq's responses in comments; if you have a computer handy, this would be an excellent moment to fire up the Rocq interpreter under your favorite IDE (see the [Preface](#) for installation instructions) and try it for yourself. Load this file, `Basics.v`, from the book's Rocq sources, find the above example, submit it to Rocq, and observe the result.)

Second, we can record what we *expect* the result to be in the form of a Rocq "example":

```
Example test_next_working_day:
  (next_working_day (next_working_day saturday)) = tuesday.
```

This declaration does two things: it makes an assertion (that the second working day after `saturday` is `tuesday`), and it gives the assertion a name that can be used to refer to it later. Having made the assertion, we can also ask Rocq to *verify* it, like this:

```
Proof. simpl. reflexivity. Qed.
```

The details are not important just now, but essentially this little script can be read as "The assertion we've just made can be proved by observing that both sides of the equality evaluate to the same thing."

Third, we can ask Rocq to *extract*, from our `Definition`, a program in a more conventional programming language (OCaml, Scheme, or Haskell) with a high-performance compiler. This facility is very useful, since it gives us a path from proved-correct algorithms written in Gallina to efficient machine code.

(Of course, we are trusting the correctness of the OCaml/Haskell/Scheme compiler, and of Rocq's extraction facility itself, but this is still a big step forward from the way most software is developed today!)

Indeed, this is one of the main uses for which Rocq was developed. We'll come back to this topic in later chapters.

The `Require Export` statement on the next line tells Rocq to use the `String` module from the standard library. We'll use strings for various things in later chapters, but we need to `Require` it here so that the grading scripts can use it for internal purposes.

```
From Stdlib Require Export String.
```

Booleans

Following the pattern of the days of the week above, we can define the standard type `bool` of booleans, with members `true` and `false`.

```
Inductive bool : Type :=
| true
| false.
```

Functions over booleans can be defined in the same way as above:

```

Definition negb (b:bool) : bool :=
  match b with
  | true  => false
  | false => true
  end.

Definition andb (b1:bool) (b2:bool) : bool :=
  match b1 with
  | true  => b2
  | false => false
  end.

Definition orb (b1:bool) (b2:bool) : bool :=
  match b1 with
  | true  => true
  | false => b2
  end.

```

(Although we are rolling our own booleans here for the sake of building up everything from scratch, Rocq does, of course, provide a default implementation of the booleans, together with a multitude of useful functions and lemmas. Wherever possible, we've named our own definitions and theorems to match the ones in the standard library.)

The last two of these illustrate Rocq's syntax for multi-argument function definitions. The corresponding multi-argument *application* syntax is illustrated by the following "unit tests," which constitute a complete specification a truth table -- for the orb function:

```

Example test_orb1: (orb true false) = true.
Proof. simpl. reflexivity. Qed.
Example test_orb2: (orb false false) = false.
Proof. simpl. reflexivity. Qed.
Example test_orb3: (orb false true) = true.
Proof. simpl. reflexivity. Qed.
Example test_orb4: (orb true true) = true.
Proof. simpl. reflexivity. Qed.

```

We can also introduce more familiar and readable infix syntax for the boolean operations we have just defined. The `Notation` command defines a new symbolic notation for an existing definition.

```

Notation "x && y" := (andb x y).
Notation "x || y" := (orb x y).

Example test_orb5: false || false || true = true.
Proof. simpl. reflexivity. Qed.

```

A note on notation: In `.v` files, we use square brackets to delimit fragments of Rocq code within comments; this convention, also used by the `rocq doc` documentation tool, keeps them visually separate from the surrounding text. In the HTML version of the files, these pieces of text appear in a different font (and without the brackets).

These examples are also an opportunity to introduce one more feature of Rocq's programming language: conditional expressions...

```

Definition negb' (b:bool) : bool :=
  if b then false
  else true.

Definition andb' (b1:bool) (b2:bool) : bool :=
  if b1 then b2
  else false.

Definition orb' (b1:bool) (b2:bool) : bool :=
  if b1 then true
  else b2.

```

Rocq's conditionals are exactly like those found in any other language, with one small generalization:

Since the `bool` type is not built in, Rocq actually supports conditional expressions over *any* inductively defined type with exactly two clauses in its definition. The guard is considered true if it evaluates to the "constructor" of the first clause of the Inductive definition (which happens to be called `true` in this case) and false if it evaluates to the second.

For example we can define the following datatype `bw`, with two constructors representing black (`b`) and white (`w`) and define a function `invert` that inverts values of this type using a conditional.

```
Inductive bw : Type :=
| bw_black
| bw_white.

Definition invert (x: bw) : bw :=
  if x then bw_white
  else bw_black.

Compute (invert bw_black).
(* ==> bw_white : bw *)

Compute (invert bw_white).
(* ==> bw_black : bw *)
```

Exercise: 1 star, standard (nandb)

The `Admitted` command can be used as a placeholder for an incomplete proof. We use it in exercises to indicate the parts that we're leaving for you -- i.e., your job is to replace `Admitted`s with real proofs.

Remove "`Admitted.`" below and complete the definition of the following function; then make sure that the `Example` assertions below can each be verified by Rocq. (i.e., fill in each proof, following the model of the `orb` tests above, and make sure Rocq accepts it.) The function should return `true` if either or both of its inputs are `false`.

Hint: if `simpl` will not simplify the goal in your proof, it's probably because you defined `nandb` without using a `match` expression. Try a different definition of `nandb`, or just skip over `simpl` and go directly to `reflexivity`. We'll explain what's happening later in the chapter.

```
Definition nandb (b1:bool) (b2:bool) : bool
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *). Admitted.

Example test_nandb1: (nandb true false) = true.
(* FILL IN HERE *) Admitted.
Example test_nandb2: (nandb false false) = true.
(* FILL IN HERE *) Admitted.
Example test_nandb3: (nandb false true) = true.
(* FILL IN HERE *) Admitted.
Example test_nandb4: (nandb true true) = false.
(* FILL IN HERE *) Admitted.
```

□

Exercise: 1 star, standard (andb3)

Do the same for the `andb3` function below. This function should return `true` when all of its inputs are `true`, and `false` otherwise.

```
Definition andb3 (b1:bool) (b2:bool) (b3:bool) : bool
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *). Admitted.

Example test_andb31: (andb3 true true true) = true.
(* FILL IN HERE *) Admitted.
Example test_andb32: (andb3 false true true) = false.
(* FILL IN HERE *) Admitted.
```

```
Example test_andb33: (andb3 true false true) = false.
(* FILL IN HERE *) Admitted.
Example test_andb34: (andb3 true true false) = false.
(* FILL IN HERE *) Admitted.
```

□

Types

Every expression in Rocq has a type describing what sort of thing it computes. The `Check` command asks Rocq to print the type of an expression.

```
Check true.
(* ==> true : bool *)
```

If the thing after `Check` is followed by a colon and a type declaration, Rocq will verify that the type of the expression matches the given type and signal an error if not.

```
Check true
: bool.
Check (negb true)
: bool.
```

Functions like `negb` itself are also data values, just like `true` and `false`. Their types are called *function types*, and they are written with arrows.

```
Check negb
: bool → bool.
```

The type of `negb`, written `bool → bool` and pronounced "bool arrow bool," can be read, "Given an input of type `bool`, this function produces an output of type `bool`." Similarly, the type of `andb`, written `bool → bool → bool`, can be read, "Given two inputs, each of type `bool`, this function produces an output of type `bool`."

New Types from Old

The types we have defined so far are examples of simple "enumerated types": their definitions explicitly enumerate a finite set of elements, called *constructors*. Here is a more interesting type definition, `color`, where one of the constructors takes an argument:

```
Inductive rgb : Type :=
| red
| green
| blue.

Inductive color : Type :=
| black
| white
| primary (p : rgb).
```

Let's look at this in a little more detail.

An `Inductive` definition does two things:

- It introduces a set of new *constructors*. E.g., `red`, `primary`, `true`, `false`, `monday`, etc. are constructors.
- It groups them into a new named type, like `bool`, `rgb`, or `color`.

Constructor expressions are formed by applying a constructor to zero or more other constructors or constructor expressions, obeying the declared number and types of the constructor arguments. E.g., these are valid constructor expressions...

- `red`
- `true`
- `primary red`
- etc.

...but these are not:

- `red primary`
- `true red`

- `primary` (`primary red`)
- etc.

In particular, the definitions of `rgb` and `color` say which constructor expressions belong to the sets `rgb` and `color`:

- `red`, `green`, and `blue` belong to the set `rgb`;
- `black` and `white` belong to the set `color`;
- if `p` is a constructor expression belonging to the set `rgb`, then `primary p` ("the constructor `primary` applied to the argument `p`") is a constructor expression belonging to the set `color`; and
- constructor expressions formed in these ways are the *only* ones belonging to the sets `rgb` and `color`.

We can define functions on colors using pattern matching just as we did for `day` and `bool`.

```
Definition monochrome (c : color) : bool :=
  match c with
  | black => true
  | white => true
  | primary p => false
  end.
```

Since the `primary` constructor takes an argument, a pattern matching `primary` should include either a variable, as we just did (note that we can choose its name freely), or a constant of appropriate type (as below).

```
Definition isred (c : color) : bool :=
  match c with
  | black => false
  | white => false
  | primary red => true
  | primary _ => false
  end.
```

The pattern "`primary _`" here is shorthand for "the constructor `primary` applied to any `rgb` constructor except `red`."

(The wildcard pattern `_` has the same effect as the dummy pattern variable `p` in the definition of `monochrome`.)

Modules

Rocq provides a *module system* to aid in organizing large developments. We won't need most of its features, but one is useful here: If we enclose a collection of declarations between `Module X` and `End X` markers, then, in the remainder of the file after the `End`, these definitions are referred to by names like `X. foo` instead of just `foo`. We will use this feature to limit the scope of definitions, so that we are free to reuse names.

```
Module Playground.
  Definition foo : rgb := blue.
End Playground.

Definition foo : bool := true.

Check Playground.foo : rgb.
Check foo : bool.
```

Tuples

```
Module TuplePlayground.
```

A single constructor with multiple parameters can be used to create a tuple type. As an example, consider representing the four bits in a nybble (half a byte). We first define a datatype `bit` that resembles `bool` (using the constructors `B0` and `B1` for the two possible bit values) and then define the datatype `nybble`, which is essentially a tuple of four bits.

```

Inductive bit : Type :=
| B1
| B0.

Inductive nybble : Type :=
| bits (b0 b1 b2 b3 : bit).

Check (bits B1 B0 B1 B0)
      : nybble.

```

The `bits` constructor acts as a wrapper for its contents. Unwrapping can be done by pattern-matching, as in the `all_zero` function below, which tests a `nybble` to see if all its bits are `B0`.

```

Definition all_zero (nb : nybble) : bool :=
  match nb with
  | (bits B0 B0 B0 B0) => true
  | (bits _ _ _ _) => false
  end.

```

(The underscore (`_`) here is a *wildcard pattern*, which avoids inventing variable names that will not be used.)

```

Compute (all_zero (bits B1 B0 B1 B0)).
(* ==> false : bool *)
Compute (all_zero (bits B0 B0 B0 B0)).
(* ==> true : bool *)

End TuplePlayground.

```

Numbers

We put this section in a module so that our own definition of natural numbers does not interfere with the one from the standard library. In the rest of the book, we'll want to use the standard library's.

```

Module NatPlayground.

```

All the types we have defined so far -- both "enumerated types" such as `day`, `bool`, and `bit` and tuple types such as `nybble` built from them -- are finite. The natural numbers, on the other hand, are an infinite set, so we'll need to use a slightly richer form of type declaration to represent them.

There are many representations of numbers to choose from. You are certainly familiar with decimal notation (base 10), using the digits 0 through 9, for example, to form the number 123. You may very likely also have encountered hexadecimal notation (base 16), in which the same number is represented as 7B, or octal (base 8), where it is 173, or binary (base 2), where it is 1111011. Using an enumerated type to represent digits, we could use any of these as our representation natural numbers. Indeed, there are circumstances where each of these choices would be useful.

The binary representation is valuable in computer hardware because the digits can be represented with just two distinct voltage levels, resulting in simple circuitry. Analogously, we wish here to choose a representation that makes *proofs* simpler.

In fact, there is a representation of numbers that is even simpler than binary, namely unary (base 1), in which only a single digit is used -- as our forebears might have done to count days by making scratches on the walls of their caves. To represent unary numbers with a Rocq datatype, we use two constructors. The capital-letter `0` constructor represents zero. The `S` constructor can be applied to the representation of the natural number `n`, yielding the representation of `n+1`, where `S` stands for "successor" (or "scratch"). Here is the complete datatype definition:

```

Inductive nat : Type :=
| 0

```

```
| S (n : nat).
```

With this definition, 0 is represented by 0, 1 by S 0, 2 by S (S 0), and so on.

Informally, the clauses of the definition can be read:

- 0 is a natural number (remember this is the letter "0," not the numeral "0").
- S can be put in front of a natural number to yield another one -- i.e., if n is a natural number, then S n is too.

Again, let's look at this a bit more closely. The definition of nat says how expressions in the set nat can be built:

- the constructor expression 0 belongs to the set nat;
- if n is a constructor expression belonging to the set nat, then S n is also a constructor expression belonging to the set nat; and
- constructor expressions formed in these two ways are the only ones belonging to the set nat.

These conditions are the precise force of the **Inductive** declaration that we gave to Rocq. They imply that the constructor expression 0, the constructor expression S 0, the constructor expression S (S 0), the constructor expression S (S (S 0)), and so on all belong to the set nat, while other constructor expressions like true, andb true false, S (S false), and 0 (0 (0 S)) do not.

A critical point here is that what we've done so far is just to define a *representation* of numbers: a way of writing them down. The names 0 and S are arbitrary, and at this point they have no special meaning -- they are just two different marks that we can use to write down numbers, together with a rule that says any nat will be written as some string of S marks followed by an 0. If we like, we can write essentially the same definition this way:

```
Inductive otherNat : Type :=  
  | stop  
  | tick (foo : otherNat).
```

The *interpretation* of these marks arises from how we use them to compute.

We can do this by writing functions that pattern match on representations of natural numbers just as we did above with booleans and days -- for example, here is the predecessor function:

```
Definition pred (n : nat) : nat :=  
  match n with  
  | 0 => 0  
  | S n' => n'  
  end.
```

The second branch can be read: "if n has the form S n' for some n', then return n'."

The following End command closes the current module, so nat will refer back to the type from the standard library.

```
End NatPlayground.
```

Because natural numbers are such a pervasive kind of data, Rocq does provide a tiny bit of built-in magic for parsing and printing them: ordinary decimal numerals can be used as an alternative to the "unary" notation defined by the constructors S and 0. Rocq prints numbers in decimal form by default:

```
Check (S (S (S (S 0)))).  
(* ==> 4 : nat *)  
  
Definition minustwo (n : nat) : nat :=  
  match n with  
  | 0 => 0  
  | S 0 => 0  
  | S (S n') => n'  
  end.
```

```
Compute (minustwo 4).
(* ==> 2 : nat *)
```

The constructor `S` has the type `nat → nat`, just like functions such as `pred` and `minustwo`:

```
Check S : nat → nat.
Check pred : nat → nat.
Check minustwo : nat → nat.
```

These are all things that can be applied to a number to yield a number. However, there is a fundamental difference between `S` and the other two: functions like `pred` and `minustwo` are defined by giving *computation rules* -- e.g., the definition of `pred` says that `pred 2` can be simplified to `1` -- while the definition of `S` has no such behavior attached. Although it is *like* a function in the sense that it can be applied to an argument, it does not *do* anything at all! It is just a way of writing down numbers.

Think about standard decimal numerals: the numeral `1` is not a computation; it's a piece of data. When we write `111` to mean the number one hundred and eleven, we are using `1`, three times, to write down a concrete representation of a number.

Let's go on and define some more functions over numbers.

For most interesting computations involving numbers, simple pattern matching is not enough: we also need recursion. For example, to check that a number `n` is even, we may need to recursively check whether `n-2` is even. Such functions are introduced with the keyword `Fixpoint` instead of `Definition`.

```
Fixpoint even (n:nat) : bool :=
  match n with
  | 0 => true
  | S 0 => false
  | S (S n') => even n'
  end.
```

We could define `odd` by a similar `Fixpoint` declaration, but here is a simpler way:

```
Definition odd (n:nat) : bool :=
  negb (even n).

Example test_odd1: odd 1 = true.
Proof. simpl. reflexivity. Qed.
Example test_odd2: odd 4 = false.
Proof. simpl. reflexivity. Qed.
```

(You may notice if you step through these proofs that `simpl` actually has no effect on the goal -- all of the work is done by `reflexivity`. We'll discuss why shortly.)

Naturally, we can also define multi-argument functions by recursion.

```
Module NatPlayground2.

Fixpoint plus (n : nat) (m : nat) : nat :=
  match n with
  | 0 => m
  | S n' => S (plus n' m)
  end.
```

Adding three to two gives us five (whew!):

```
Compute (plus 3 2).
(* ==> 5 : nat *)
```

The steps of simplification that Rocq performs here can be visualized as follows:

```
(*      plus 3 2
   i.e. plus (S (S (S 0))) (S (S 0))
   ==> S (plus (S (S 0)) (S (S 0)))
        by the second clause of the match
   ==> S (S (plus (S 0) (S (S 0))))
        by the second clause of the match
   ==> S (S (S (plus 0 (S (S 0)))))
        by the second clause of the match
```

```

=> S (S (S (S (S 0))))
      by the first clause of the match
i.e. 5 *)

```

As a notational convenience, if two or more arguments have the same type, they can be grouped together. In the following definition, $(n\ m : \text{nat})$ means just the same as if we had written $(n : \text{nat})\ (m : \text{nat})$.

```

Fixpoint mult (n m : nat) : nat :=
  match n with
  | 0 => 0
  | S n' => plus m (mult n' m)
  end.

```

Example `test_mult1`: `(mult 3 3) = 9`.
 Proof. `simpl. reflexivity. Qed.`

We can match two expressions at once by putting a comma between them:

```

Fixpoint minus (n m:nat) : nat :=
  match n, m with
  | 0, _ => 0
  | S _, 0 => n
  | S n', S m' => minus n' m'
  end.

```

End `NatPlayground2`.

```

Fixpoint exp (base power : nat) : nat :=
  match power with
  | 0 => S 0
  | S p => mult base (exp base p)
  end.

```

Exercise: 1 star, standard (factorial)

Recall the standard mathematical factorial function:

$$\begin{aligned} \text{factorial}(0) &= 1 \\ \text{factorial}(n) &= n * \text{factorial}(n-1) \quad (\text{if } n > 0) \end{aligned}$$

Translate this into Rocq.

Hint: Make sure you put a `:=` between the header we've provided and your definition. If you see an error like "The reference factorial was not found in the current environment," it means you've forgotten the `:=`.

```

Fixpoint factorial (n:nat) : nat
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *). Admitted.

```

```

Example test_factorial1: (factorial 3) = 6.
(* FILL IN HERE *) Admitted.
Example test_factorial2: (factorial 5) = (mult 10 12).
(* FILL IN HERE *) Admitted.

```

□

Again, we can make numerical expressions easier to read and write by introducing notations for addition, subtraction, and multiplication.

```

Notation "x + y" := (plus x y)
                    (at level 50, left associativity)
                    : nat_scope.
Notation "x - y" := (minus x y)
                    (at level 50, left associativity)
                    : nat_scope.
Notation "x * y" := (mult x y)
                    (at level 40, left associativity)
                    : nat_scope.

```

```

Check ((0 + 1) + 1) : nat.

```

(The `level`, `associativity`, and `nat_scope` annotations control how these notations are treated by Rocq's parser. The details are not important for present purposes, but

interested readers can check out the "More on Notation" section at the end of this chapter.)

Note that these declarations do not change the definitions we've already made: they are simply instructions to Rocq's parser to accept `x + y` in place of `plus x y` and, conversely, to its pretty-printer to display `plus x y` as `x + y`.

When we say that Rocq comes with almost nothing built-in, we really mean it: even testing equality is a user-defined operation! Here is a function `eqb` that tests natural numbers for equality, yielding a boolean. Note the use of nested matches -- we could also have used a simultaneous match, as in `minus`.

```
Fixpoint eqb (n m : nat) : bool :=
  match n with
  | 0 => match m with
        | 0 => true
        | S m' => false
      end
  | S n' => match m with
            | 0 => false
            | S m' => eqb n' m'
          end
  end.
```

Similarly, the `leb` function tests whether its first argument is less than or equal to its second argument, yielding a boolean.

```
Fixpoint leb (n m : nat) : bool :=
  match n with
  | 0 => true
  | S n' =>
    match m with
    | 0 => false
    | S m' => leb n' m'
    end
  end.
```

```
Example test_leb1: leb 2 2 = true.
Proof. simpl. reflexivity. Qed.
Example test_leb2: leb 2 4 = true.
Proof. simpl. reflexivity. Qed.
Example test_leb3: leb 4 2 = false.
Proof. simpl. reflexivity. Qed.
```

We'll be using these (especially `eqb`) a lot, so let's give them infix notations.

```
Notation "x =? y" := (eqb x y) (at level 70) : nat_scope.
Notation "x <=? y" := (leb x y) (at level 70) : nat_scope.

Example test_leb3': (4 <=? 2) = false.
Proof. simpl. reflexivity. Qed.
```

We now have two symbols that both look like equality: `=` and `=?`. We'll have much more to say about their differences and similarities later. For now, the main thing to notice is that `x = y` is a logical *claim* -- a "proposition" -- that we can try to prove, while `x =? y` is a boolean *expression* whose value (either `true` or `false`) we can compute.

Exercise: 1 star, standard (ltb)

Fill in the definition of an `ltb` function that tests natural numbers for less-than, yielding a boolean.

Hint: Instead of making up a new `Fixpoint` for this one, define it in terms of a previously defined function. It can be done with just one previously defined function, but you can use two if you want.

```
Definition ltb (n m : nat) : bool
  (* REPLACE THIS LINE WITH " := _your_definition_ ." *). Admitted.

Notation "x <? y" := (ltb x y) (at level 70) : nat_scope.
```

```

Example test_ltb1: (ltb 2 2) = false.
(* FILL IN HERE *) Admitted.
Example test_ltb2: (ltb 2 4) = true.
(* FILL IN HERE *) Admitted.
Example test_ltb3: (ltb 4 2) = false.
(* FILL IN HERE *) Admitted.

```

Proof by Simplification

Now that we've looked at a few datatypes and functions, let's turn to stating and proving properties of their behavior.

Actually, we've already started doing this: each `Example` in the previous sections made a precise claim about the behavior of some function on some particular inputs. The proofs of these claims were always the same: use `simpl` to simplify both sides of the equation, then use `reflexivity` to check that both sides contain identical values.

```

Example plus_1_1 : 1 + 1 = 2.
Proof. simpl. reflexivity. Qed.

```

The same sort of "proof by simplification" can also be used to establish more interesting properties. For example, the fact that `0` is a "neutral element" for `+` on the left can be proved just by observing that `0 + n` reduces to `n` no matter what `n` is -- a fact that can be read off directly from the definition of `plus`.

```

Theorem plus_0_n : ∀ n : nat, 0 + n = n.
Proof.
  intros n. simpl. reflexivity. Qed.

```

(You may notice that the above statement looks different if you look at the `.v` file in your IDE than it does if you view the HTML rendition in your browser. In `.v` files, we write the universal quantifier `∀` using the reserved identifier "forall." When the `.v` files are converted to HTML, this gets transformed into the standard upside-down-A symbol.)

This is a good place to mention that `reflexivity` is a bit more powerful than we have suggested. In the examples we have seen, the calls to `simpl` were actually not required because `reflexivity` will do some simplification automatically when checking that two sides are equal; `simpl` was just added so that we could see the intermediate state, after simplification but before finishing the proof. Here is a shorter proof:

```

Theorem plus_0_n' : ∀ n : nat, 0 + n = n.
Proof.
  intros n. reflexivity. Qed.

```

Moreover, it is useful to know that `reflexivity` does somewhat *more* simplification than `simpl` does -- for example, it tries "unfolding" defined terms, replacing them with their right-hand sides. The reason for this difference is that, if `reflexivity` succeeds, the whole goal is finished and we don't need to look at whatever expanded expressions `reflexivity` has created by all this simplification and unfolding; by contrast, `simpl` is used in situations where we may have to read and understand the new goal that it creates, so we would not want it blindly expanding definitions and leaving the goal in a messy state.

The form of the theorem we just stated and its proof are almost exactly the same as the simpler examples we saw earlier; there are just a few differences.

First, we've used the keyword `Theorem` instead of `Example`. This difference is just a matter of style; the keywords `Example` and `Theorem` (and a few others, including `Lemma`, `Fact`, and `Remark`) mean pretty much the same thing to Rocq.

Second, we've added the quantifier `∀ n : nat`, so that our theorem talks about *all* natural numbers `n`. Informally, to prove theorems of this form, we generally start by saying "Suppose

n is some number..." Formally, this is achieved in the proof by `intros n`, which moves n from the quantifier in the goal to a *context* of current assumptions.

Incidentally, we could have used another identifier instead of n in the `intros` clause, (though of course this might be confusing to human readers of the proof):

```
Theorem plus_0_n'' : ∀ n : nat, 0 + n = n.  
Proof.  
  intros m. reflexivity. Qed.
```

The keywords `intros`, `simpl`, and `reflexivity` are examples of *tactics*. A tactic is a command that is used between `Proof` and `Qed` to guide the process of checking some claim we are making. We will see several more tactics in the rest of this chapter and many more in future chapters.

Other similar theorems can be proved with the same pattern.

```
Theorem plus_1_l : ∀ n:nat, 1 + n = S n.  
Proof.  
  intros n. reflexivity. Qed.  
  
Theorem mult_0_l : ∀ n:nat, 0 × n = 0.  
Proof.  
  intros n. reflexivity. Qed.
```

The `_l` suffix in the names of these theorems is pronounced "on the left."

It is worth stepping through these proofs to observe how the context and the goal change. You may want to add calls to `simpl` before `reflexivity` to see the simplifications that Rocq performs on the terms before checking that they are equal.

Proof by Rewriting

The following theorem is a bit more interesting than the ones we've seen:

```
Theorem plus_id_example : ∀ n m:nat,  
  n = m →  
  n + n = m + m.
```

Instead of making a universal claim about all numbers n and m , it talks about a more specialized property that only holds when $n = m$. The arrow symbol is pronounced "implies."

As before, we need to be able to reason by assuming we are given such numbers n and m . We also need to assume the hypothesis $n = m$. The `intros` tactic will serve to move all three of these from the goal into assumptions in the current context.

Since n and m are arbitrary numbers, we can't just use simplification to prove this theorem. Instead, we prove it by observing that, if we are assuming $n = m$, then we can replace n with m in the goal statement and obtain an equality with the same expression on both sides. The tactic that tells Rocq to perform this replacement is called `rewrite`.

```
Proof.  
  (* move both quantifiers into the context: *)  
  intros n m.  
  (* move the hypothesis into the context: *)  
  intros H.  
  (* rewrite the goal using the hypothesis: *)  
  rewrite → H.  
  reflexivity. Qed.
```

The first line of the proof moves the universally quantified variables n and m into the context. The second moves the hypothesis $n = m$ into the context and gives it the name H . The third tells Rocq to rewrite the current goal ($n + n = m + m$) by replacing the left side of the equality hypothesis H with the right side.

(The arrow symbol in the `rewrite` has nothing to do with implication: it tells Rocq to apply the rewrite from left to right. Indeed, we can omit the arrow, and Rocq will default to rewriting left to right. To rewrite from right to left, use `rewrite <-`. Try making this change in the above proof and see what changes.)

Exercise: 1 star, standard (plus id exercise)

Remove "Admitted." and fill in the proof. (Note that the theorem has *two* hypotheses -- $n = m$ and $m = 0$ -- each to the left of an implication arrow.)

```
Theorem plus_id_exercise : ∀ n m o : nat,
  n = m → m = 0 → n + m = m + 0.
Proof.
  (* FILL IN HERE *) Admitted.
□
```

The `Admitted` command tells Rocq that we want to skip trying to prove this theorem and just accept it as a given. This is often useful for developing longer proofs: we can state subsidiary lemmas that we believe will be useful for making some larger argument, use `Admitted` to accept them on faith for the moment, and continue working on the main argument until we are sure it makes sense; then we can go back and fill in the proofs we skipped.

Be careful, though: every time you say `Admitted` you are leaving a door open for total nonsense to enter Rocq's safe, formally checked world!

The `Check` command can also be used to examine the statements of previously declared lemmas and theorems. The two examples below are lemmas about multiplication that are proved in the standard library. (We will see how to prove them ourselves in the next chapter.)

```
Check mult_n_0.
(* ==> forall n : nat, 0 = n * 0 *)

Check mult_n_Sm.
(* ==> forall n m : nat, n * m + n = n * S m *)
```

We can use the `rewrite` tactic with a previously proved theorem instead of a hypothesis from the context. If the statement of the previously proved theorem involves quantified variables, as in the example below, Rocq will try to fill in appropriate values for them by matching the body of the previous theorem statement against the current goal.

```
Theorem mult_n_0_m_0 : ∀ p q : nat,
  (p × 0) + (q × 0) = 0.
Proof.
  intros p q.
  rewrite <- mult_n_0.
  rewrite <- mult_n_0.
  reflexivity. Qed.
```

Exercise: 1 star, standard (mult n 1)

Use `mult_n_Sm` and `mult_n_0` to prove the following theorem. (Recall that `1` is `S 0`.)

```
Theorem mult_n_1 : ∀ p : nat,
  p × 1 = p.
Proof.
  (* FILL IN HERE *) Admitted.
□
```

Proof by Case Analysis

Of course, not everything can be proved by simple calculation and rewriting: In general, unknown, hypothetical values (arbitrary numbers, booleans, lists, etc.) can block simplification. For example, if we try to prove the following fact using the `simpl` tactic as above, we get stuck. (We then use the `Abort` command to give up on it for the moment.)

```

Theorem plus_1_neq_0_firsttry : ∀ n : nat,
  (n + 1) =? 0 = false.
Proof.
  intros n.
  simpl. (* does nothing! *)
Abort.

```

The reason for getting stuck is that the definitions of both `eqb` and `+` begin by performing a `match` on their first argument. Here, the first argument to `+` is the unknown number `n` and the argument to `eqb` is the compound expression `n + 1`; neither can be simplified.

To make progress, we need to consider the possible forms of `n`, separately. If `n` is `0`, then we can calculate the final result of `(n + 1) =? 0` and check that it is, indeed, `false`. And if `n = S n'` for some `n'`, then -- although we don't know exactly what number `n + 1` represents -- we can calculate that at least it will begin with one `S`; and this is enough to calculate that, again, `(n + 1) =? 0` will yield `false`.

The tactic that tells Rocq to consider separate cases where `n = 0` and `n = S n'` is called `destruct`.

```

Theorem plus_1_neq_0 : ∀ n : nat,
  (n + 1) =? 0 = false.
Proof.
  intros n. destruct n as [| n'] eqn:E.
  - reflexivity.
  - reflexivity. Qed.

```

The `destruct` generates *two* subgoals, which we must then prove, separately, in order to get Rocq to accept the theorem.

The annotation "`as [| n']`" is called an *intro pattern*. It tells Rocq what variable names to introduce in each subgoal. In general, what goes between the square brackets is a *list of lists* of names, separated by `|`. In this case, the first component is empty, since the `0` constructor doesn't take any arguments. The second component gives a single name, `n'`, since `S` is a unary constructor.

In each subgoal, Rocq remembers the assumption about `n` that is relevant for this subgoal -- either `n = 0` or `n = S n'` for some `n'`. The `eqn:E` annotation tells `destruct` to give the name `E` to this equation. (Leaving off the `eqn:E` annotation causes Rocq to elide these assumptions in the subgoals. This slightly streamlines proofs where the assumptions are not explicitly used, but it is better practice to keep them for the sake of documentation, as they can help keep you oriented when working with the subgoals.)

The `-` signs on the second and third lines are called *bullets*, and they mark the parts of the proof that correspond to the two generated subgoals. The part of the proof script that comes after a bullet is the entire proof for the corresponding subgoal. In this example, each of the subgoals is easily proved by a single use of `reflexivity`, which itself performs some simplification -- e.g., the second one simplifies `(S n' + 1) =? 0` to `false` by first rewriting `(S n' + 1)` to `S (n' + 1)`, then unfolding `eqb`, and then simplifying the `match`.

Marking cases with bullets is optional: if bullets are not present, Rocq simply expects you to prove each subgoal in sequence, one at a time. But it is a good idea to use bullets, and you should make a habit of doing it! For one thing, bullets make the structure of a proof apparent, improving readability. Moreover, bullets instruct Rocq to ensure that a subgoal is complete before trying to verify the next one, preventing proofs for different subgoals from getting mixed up. These issues become especially important in larger developments, where fragile proofs can lead to long debugging sessions!

There are no hard and fast rules for how proofs should be formatted in Rocq -- e.g., where lines should be broken and how sections of the proof should be indented to indicate their nested structure. However, if the places where multiple subgoals are generated are marked with explicit bullets at the beginning of lines, then the proof will be readable almost no matter what choices are made about other aspects of layout.

This is also a good place to mention one other piece of somewhat obvious advice about line lengths. Beginning Rocq users sometimes tend to the extremes, either writing each tactic on its own line or writing entire proofs on a single line. Good style lies somewhere in the middle. One reasonable guideline is to limit yourself to 80 (or, if you have a wide screen or good eyes, 120) character lines.

The `destruct` tactic can be used with any inductively defined datatype. For example, we use it next to prove that boolean negation is involutive -- i.e., that negation is its own inverse.

```
Theorem negb_involutive : ∀ b : bool,
  negb (negb b) = b.
Proof.
  intros b. destruct b eqn:E.
  - reflexivity.
  - reflexivity. Qed.
```

Note that the `destruct` here has no `as` clause because none of the subcases of the `destruct` need to bind any variables, so there is no need to specify any names. In fact, we can omit the `as` clause from *any* `destruct` and Rocq will fill in variable names automatically. This is generally considered bad style, since Rocq often makes confusing choices of names when left to its own devices.

It is sometimes useful to invoke `destruct` inside a subgoal, generating yet more proof obligations. In this case, we use different kinds of bullets to mark goals on different "levels." For example:

```
Theorem andb_commutative : ∀ b c, andb b c = andb c b.
Proof.
  intros b c. destruct b eqn:Eb.
  - destruct c eqn:Ec.
    + reflexivity.
    + reflexivity.
  - destruct c eqn:Ec.
    + reflexivity.
    + reflexivity.
Qed.
```

Each pair of calls to `reflexivity` corresponds to the subgoals that were generated after the execution of the `destruct c` line right above it.

Besides `-` and `+`, we can use `×` (asterisk) or any repetition of a bullet symbol (e.g. `--` or `***`) as a bullet. We can also enclose sub-proofs in curly braces instead of using bullets:

```
Theorem andb_commutative' : ∀ b c, andb b c = andb c b.
Proof.
  intros b c. destruct b eqn:Eb.
  { destruct c eqn:Ec.
    { reflexivity. }
    { reflexivity. } }
  { destruct c eqn:Ec.
    { reflexivity. }
    { reflexivity. } }
Qed.
```

Since curly braces mark both the beginning and the end of a proof, they can be used for multiple subgoal levels, as this example shows. Furthermore, curly braces allow us to reuse the same bullet shapes at multiple levels in a proof.

The choice of braces, bullets, or a combination of the two is purely a matter of taste.

```
Theorem andb3_exchange :
  ∀ b c d, andb (andb b c) d = andb (andb b d) c.
Proof.
  intros b c d. destruct b eqn:Eb.
  - destruct c eqn:Ec.
    { destruct d eqn:Ed.
      - reflexivity.
      - reflexivity. }
  { destruct d eqn:Ed.
```

```

- reflexivity.
- reflexivity. }
- destruct c eqn:Ec.
{ destruct d eqn:Ed.
- reflexivity.
- reflexivity. }
{ destruct d eqn:Ed.
- reflexivity.
- reflexivity. }

```

Qed.

Exercise: 2 stars, standard (andb true elim2)

Prove the following claim, marking cases (and subcases) with bullets when you use `destruct`.

Hint 1: Thus far, we've only ever used `simpl` to simplify the goal. It is also possible to use it to simplify hypotheses: use `simpl in H`, where `H` is a hypothesis, to simplify it. You may find this useful in this exercise.

Hint 2: You will eventually need to destruct both booleans, as in the theorems above. It is better to simplify the hypothesis before attempting to destruct the second boolean.

Hint 3: If you reach a contradiction in the hypotheses, focus on how to `rewrite` using that contradiction.

```

Theorem andb_true_elim2 : ∀ b c : bool,
  andb b c = true → c = true.
Proof.
  (* FILL IN HERE *) Admitted.

```

□

Before closing the chapter, we should mention one final convenience. As you may have noticed, many proofs perform case analysis on a variable right after introducing it:

```
intros x y. destruct y as [|y] eqn:E.
```

This pattern is so common that Rocq provides a shorthand for it: we can perform case analysis on a variable when introducing it by using an intro pattern instead of a variable name. For instance, here is a shorter proof of the `plus_1_neq_0` theorem above. (You'll also note one downside of this shorthand: we lose the equation recording the assumption we are making in each subgoal, which we previously got from the `eqn:E` annotation.)

```

Theorem plus_1_neq_0' : ∀ n : nat,
  (n + 1) =? 0 = false.
Proof.
  intros [|n].
  - reflexivity.
  - reflexivity. Qed.

```

If there are no constructor arguments that need names, we can just write `[]` to get the case analysis.

```

Theorem andb_commutative'' :
  ∀ b c, andb b c = andb c b.
Proof.
  intros [].
  - reflexivity.
  - reflexivity.
  - reflexivity.
  - reflexivity.
Qed.

```

Exercise: 1 star, standard (zero nbeq_plus 1)

```

Theorem zero_nbeq_plus_1 : ∀ n : nat,
  0 =? (n + 1) = false.
Proof.
  (* FILL IN HERE *) Admitted.

```

□

More on Notation (Optional)

(In general, sections marked Optional are not needed to follow the rest of the book, except possibly other Optional sections. On a first reading, you might want to just skim these sections so that you know what's there for future reference.)

Recall the notation definitions for infix plus and times:

```
Notation "x + y" := (plus x y)
                    (at level 50, left associativity)
                    : nat_scope.
Notation "x * y" := (mult x y)
                    (at level 40, left associativity)
                    : nat_scope.
```

For each notation symbol in Rocq, we can specify its *precedence level* and its *associativity*. The precedence level n is specified by writing `at level  $n$` ; this helps Rocq parse compound expressions. The associativity setting helps to disambiguate expressions containing multiple occurrences of the same symbol. For example, the parameters specified above for `+` and `*` say that the expression `1+2*3*4` is shorthand for `(1+((2*3)*4))`. Rocq uses precedence levels from 0 to 100, and *left*, *right*, or *no* associativity. We will see more examples of this later, e.g., in the [Lists](#) chapter.

Each notation symbol is also associated with a *notation scope*. Rocq tries to guess what scope is meant from context, so when it sees `S (0×0)` it guesses `nat_scope`, but when it sees the pair type `bool×bool` (which we'll see in a later chapter) it guesses `type_scope`. Occasionally, it is necessary to help it out by writing, for example, `(x×y)%nat`, and sometimes in what Rocq prints it will use `%nat` to indicate what scope a notation is in.

Notation scopes also apply to numeral notations (3, 4, 5, 42, etc.), so you may sometimes see `0%nat`, which means 0 (the natural number 0 that we're using in this chapter), or `0%Z`, which means the integer zero (which comes from a different part of the standard library).

Pro tip: Rocq's notation mechanism is not especially powerful. Don't expect too much from it.

Fixpoints and Structural Recursion (Optional)

Here is a copy of the definition of addition:

```
Fixpoint plus' (n : nat) (m : nat) : nat :=
  match n with
  | 0 => m
  | S n' => S (plus' n' m)
  end.
```

When Rocq checks this definition, it notes that `plus'` is "decreasing on 1st argument." What this means is that we are performing a *structural recursion* over the argument `n` -- i.e., that we make recursive calls only on strictly smaller values of `n`. This implies that all calls to `plus'` will eventually terminate. Rocq demands that some argument of *every* Fixpoint definition be "decreasing."

This requirement is a fundamental feature of Rocq's design: In particular, it guarantees that every function that can be defined in Rocq will terminate on all inputs. However, because Rocq's "decreasing analysis" is not very sophisticated, it is sometimes necessary to write functions in slightly unnatural ways.

Exercise: 2 stars, standard, optional (decreasing)

To get a concrete sense of this, find a way to write a sensible Fixpoint definition (of a simple function on numbers, say) that *does* terminate on all inputs, but that Rocq will reject because of this restriction.

(If you choose to turn in this optional exercise as part of a homework assignment, make sure you comment out your solution so that it doesn't cause Rocq to reject the whole file!)

```
(* FILL IN HERE *)
```

□

More Exercises

Warmups

Exercise: 1 star, standard (identity_fn_applied_twice)

Use the tactics you have learned so far to prove the following theorem about boolean functions.

```
Theorem identity_fn_applied_twice :  
  ∀ (f : bool → bool),  
  (∀ (x : bool), f x = x) →  
  ∀ (b : bool), f (f b) = b.  
Proof.  
  (* FILL IN HERE *) Admitted.
```

□

Exercise: 1 star, standard (negation_fn_applied_twice)

Now state and prove a theorem `negation_fn_applied_twice` similar to the previous one but where the hypothesis says that the function `f` has the property that `f x = negb x`.

```
(* FILL IN HERE *)
```

```
(* Do not modify the following line: *)  
Definition manual_grade_for_negation_fn_applied_twice : option (nat×string)  
:= None.
```

(The last definition is used by the autograder.) □

Exercise: 3 stars, standard, optional (andb_eq_orb)

Prove the following theorem. (Hint: This can be a bit tricky, depending on how you approach it. You will probably need both `destruct` and `rewrite`, but destructing everything in sight is not the best way.)

```
Theorem andb_eq_orb :  
  ∀ (b c : bool),  
  (andb b c = orb b c) →  
  b = c.  
Proof.  
  (* FILL IN HERE *) Admitted.
```

□

Course Late Policies, Formalized

Suppose that a course has a grading policy based on late days, where a student's final letter grade is lowered if they submit too many homework assignments late.

In the next series of problems, we model this situation using the features of Rocq that we have seen so far and prove some simple facts about this grading policy.

```
Module LateDays.
```

First, we introduce a datatype for modeling the "letter" component of a grade.

```
Inductive letter : Type :=  
  | A | B | C | D | F.
```

Then we define the modifiers -- a `Natural A` is just a "plain" grade of A.

```
Inductive modifier : Type :=  
  | Plus | Natural | Minus.
```

A full grade, then, is just a letter and a modifier.

We might write, informally, "A-" for the Rocq value Grade A Minus, and similarly "C" for the Rocq value Grade C Natural.

```
Inductive grade : Type :=  
  Grade (l:letter) (m:modifier).
```

We will want to be able to say when one grade is "better" than another. In other words, we need a way to compare two grades. As with natural numbers, we could define `bool`-valued functions `grade_eqb`, `grade_ltb`, etc., and that would work fine. However, we can also define a slightly more informative type for comparing two values, as shown below. This datatype has three constructors that can be used to indicate whether two values are "equal", "less than", or "greater than" one another. (This definition also appears in the Rocq standard library.)

```
Inductive comparison : Type :=  
  | Eq (* "equal" *)  
  | Lt (* "less than" *)  
  | Gt. (* "greater than" *)
```

Using pattern matching, it is not difficult to define the comparison operation for two letters l_1 and l_2 (see below). This definition uses two features of `match` patterns: First, recall that we can match against *two* values simultaneously by separating them and the corresponding patterns with comma `,`. This is simply a convenient abbreviation for nested pattern matching. For example, the match expression on the left below is just shorthand for the lower-level "expanded version" shown on the right:

<pre>match l₁, l₂ with A, A ⇒ Eq A, _ ⇒ Gt end</pre>	<pre>match l₁ with A ⇒ match l₂ with A ⇒ Eq _ ⇒ Gt end end</pre>
--	--

As another shorthand, we can also match one of several possibilities by using `|` in the pattern. For example the pattern `C, (A | B)` stands for two cases: `C, A` and `C, B`.

```
Definition letter_comparison (l1 l2 : letter) : comparison :=  
  match l1, l2 with  
  | A, A ⇒ Eq  
  | A, _ ⇒ Gt  
  | B, A ⇒ Lt  
  | B, B ⇒ Eq  
  | B, _ ⇒ Gt  
  | C, (A | B) ⇒ Lt  
  | C, C ⇒ Eq  
  | C, _ ⇒ Gt  
  | D, (A | B | C) ⇒ Lt  
  | D, D ⇒ Eq  
  | D, _ ⇒ Gt  
  | F, (A | B | C | D) ⇒ Lt  
  | F, F ⇒ Eq  
  end.
```

We can test the `letter_comparison` operation by trying it out on a few examples.

```
Compute letter_comparison B A.
```

`==> Lt`

```
Compute letter_comparison D D.
```

`==> Eq`

```
Compute letter_comparison B F.
```

==> Gt

As a further sanity check, we can prove that the `letter_comparison` function does indeed give the result `Eq` when comparing a letter `l` against itself.

Exercise: 1 star, standard (letter_comparison)

```
Theorem letter_comparison_Eq :  
  ∀ l, letter_comparison l l = Eq.  
Proof.  
  (* FILL IN HERE *) Admitted.  
□
```

We can follow the same strategy to define the comparison operation for two grade modifiers. We consider them to be ordered as `Plus > Natural > Minus`.

```
Definition modifier_comparison (m1 m2 : modifier) : comparison :=  
  match m1, m2 with  
  | Plus, Plus => Eq  
  | Plus, _ => Gt  
  | Natural, Plus => Lt  
  | Natural, Natural => Eq  
  | Natural, _ => Gt  
  | Minus, (Plus | Natural) => Lt  
  | Minus, Minus => Eq  
end.
```

Exercise: 2 stars, standard (grade_comparison)

Use pattern matching to complete the following definition.

(This ordering on grades is sometimes called "lexicographic" ordering: we first compare the letters, and we only consider the modifiers in the case that the letters are equal. I.e. all grade variants of A are greater than all grade variants of B.)

Hint: match against `g1` and `g2` simultaneously, but don't try to enumerate all the cases.

Instead do case analysis on the result of a suitable call to `letter_comparison` to end up with just 3 possibilities.

```
Definition grade_comparison (g1 g2 : grade) : comparison  
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *). Admitted.
```

The following "unit tests" of your `grade_comparison` function should pass once you have defined it correctly.

```
Example test_grade_comparison1 :  
  (grade_comparison (Grade A Minus) (Grade B Plus)) = Gt.  
  (* FILL IN HERE *) Admitted.  
  
Example test_grade_comparison2 :  
  (grade_comparison (Grade A Minus) (Grade A Plus)) = Lt.  
  (* FILL IN HERE *) Admitted.  
  
Example test_grade_comparison3 :  
  (grade_comparison (Grade F Plus) (Grade F Plus)) = Eq.  
  (* FILL IN HERE *) Admitted.  
  
Example test_grade_comparison4 :  
  (grade_comparison (Grade B Minus) (Grade C Plus)) = Gt.  
  (* FILL IN HERE *) Admitted.  
□
```

Now that we have a definition of grades and how they compare to one another, let us implement a late-penalty function.

First, we define what it means to lower the `letter` component of a grade. Since F is already the lowest grade possible, we just leave it alone.

```

Definition lower_letter (l : letter) : letter :=
  match l with
  | A => B
  | B => C
  | C => D
  | D => F
  | F => F (* Can't go lower than F! *)
  end.

```

Our formalization can already help us understand some corner cases of the grading policy. For example, we might expect that if we use the `lower_letter` function its result will actually be lower, as claimed in the following theorem. But this theorem is not provable! (Do you see why?)

```

Theorem lower_letter_lowers: ∀ (l : letter),
  letter_comparison (lower_letter l) l = Lt.
Proof.
  intros l.
  destruct l.
  - simpl. reflexivity.
  - simpl. reflexivity.
  - simpl. reflexivity.
  - simpl. reflexivity.
  - simpl. (* We get stuck here. *)
Abort.

```

The problem, of course, has to do with the "edge case" of lowering F, as we can see like this:

```

Theorem lower_letter_F_is_F:
  lower_letter F = F.
Proof.
  simpl. reflexivity.
Qed.

```

With this insight, we can state a better version of the lower letter theorem that actually is provable. In this version, the hypothesis about F says that F is strictly smaller than `l`, which rules out the problematic case above. In other words, as long as `l` is bigger than F, it will be lowered.

Exercise: 2 stars, standard (lower_letter_lowers)

Prove the following theorem.

```

Theorem lower_letter_lowers:
  ∀ (l : letter),
    letter_comparison F l = Lt →
    letter_comparison (lower_letter l) l = Lt.
Proof.
  (* FILL IN HERE *) Admitted.
□

```

Exercise: 2 stars, standard (lower_grade)

We can now use the `lower_letter` definition as a helper to define what it means to lower a grade by one step. Complete the definition below so that it sends a grade `g` to one step lower (unless it is already Grade F Minus, which should remain unchanged). Once you have implemented it correctly, the subsequent "unit test" examples should hold trivially.

Hint: To make this a succinct definition that is easy to prove properties about, you will probably want to use nested pattern matching. The outer match should not match on the specific letter component of the grade -- it should consider only the modifier. You should definitely *not* try to enumerate all of the cases.

Our solution is under 10 lines of code total.

```

Definition lower_grade (g : grade) : grade
  (* REPLACE THIS LINE WITH " := _your_definition_ ." *). Admitted.

Example lower_grade_A_Plus :
  lower_grade (Grade A Plus) = (Grade A Natural).

```

```

Proof.
(* FILL IN HERE *) Admitted.

Example lower_grade_A_Natural :
  lower_grade (Grade A Natural) = (Grade A Minus).
Proof.
(* FILL IN HERE *) Admitted.

Example lower_grade_A_Minus :
  lower_grade (Grade A Minus) = (Grade B Plus).
Proof.
(* FILL IN HERE *) Admitted.

Example lower_grade_B_Plus :
  lower_grade (Grade B Plus) = (Grade B Natural).
Proof.
(* FILL IN HERE *) Admitted.

Example lower_grade_F_Natural :
  lower_grade (Grade F Natural) = (Grade F Minus).
Proof.
(* FILL IN HERE *) Admitted.

Example lower_grade_twice :
  lower_grade (lower_grade (Grade B Minus)) = (Grade C Natural).
Proof.
(* FILL IN HERE *) Admitted.

Example lower_grade_thrice :
  lower_grade (lower_grade (lower_grade (Grade B Minus))) = (Grade C Minus).
Proof.
(* FILL IN HERE *) Admitted.

```

Rocq makes no distinction between an `Example` and a `Theorem`. We state the following as a `Theorem` only as a hint that we will use it in proofs below.

```

Theorem lower_grade_F_Minus : lower_grade (Grade F Minus) = (Grade F Minus).
Proof.
(* FILL IN HERE *) Admitted.

(* GRADE_THEOREM 0.25: lower_grade_A_Plus *)
(* GRADE_THEOREM 0.25: lower_grade_A_Natural *)
(* GRADE_THEOREM 0.25: lower_grade_A_Minus *)
(* GRADE_THEOREM 0.25: lower_grade_B_Plus *)
(* GRADE_THEOREM 0.25: lower_grade_F_Natural *)
(* GRADE_THEOREM 0.25: lower_grade_twice *)
(* GRADE_THEOREM 0.25: lower_grade_thrice *)
(* GRADE_THEOREM 0.25: lower_grade_F_Minus *)

```

□

Exercise: 3 stars, standard (lower grade lowers)

Prove the following theorem, which says that, as long as the grade starts out above F-, the `lower_grade` option does indeed lower the grade. As usual, destructing everything in sight is *not* a good idea. Judicious use of `destruct` along with rewriting is a better strategy.

Hint: If you defined your `grade_comparison` function as suggested, you will only need to destruct a `letter` in one case. The case for F will probably benefit from `lower_grade_F_Minus`.

```

Theorem lower_grade_lowers :
  ∀ (g : grade),
    grade_comparison (Grade F Minus) g = Lt →
    grade_comparison (lower_grade g) g = Lt.
Proof.
(* FILL IN HERE *) Admitted.

```

□

Now that we have implemented and tested a function that lowers a grade by one step, we can implement a specific late-days policy. Given a number of `late_days`, the

`apply_late_policy` function computes the final grade from `g`, the initial grade.

This function encodes the following policy:

# late days	penalty
0 - 8	no penalty
9 - 16	lower grade by one step (A+ => A, A => A-, A- => B+, etc.)
17 - 20	lower grade by two steps
>= 21	lower grade by three steps (a whole letter)

```
Definition apply_late_policy (late_days : nat) (g : grade) : grade :=
  if late_days <? 9 then g
  else if late_days <? 17 then lower_grade g
  else if late_days <? 21 then lower_grade (lower_grade g)
  else lower_grade (lower_grade (lower_grade g)).
```

Sometimes it is useful to be able to "unfold" a definition to be able to make progress on a proof. Soon, we will see how to do this in a much simpler way automatically, but for now, it is easy to prove that a use of any definition like `apply_late_policy` is equal to its right hand side just by using reflexivity.

This result is useful because it allows us to use `rewrite` to expose the internals of the definition.

```
Theorem apply_late_policy_unfold :
  ∀ (late_days : nat) (g : grade),
    (apply_late_policy late_days g)
    =
    (if late_days <? 9 then g else
     if late_days <? 17 then lower_grade g
     else if late_days <? 21 then lower_grade (lower_grade g)
     else lower_grade (lower_grade (lower_grade g))).
Proof.
  intros. reflexivity.
Qed.
```

Now let's prove some properties about this policy.

The next theorem states that if a student accrues no more than eight late days throughout the semester, their grade is unaffected. It is easy to prove: once you use the `apply_late_policy_unfold` you can rewrite using the hypothesis.

Exercise: 2 stars, standard (no penalty for mostly on time)

```
Theorem no_penalty_for_mostly_on_time :
  ∀ (late_days : nat) (g : grade),
    (late_days <? 9 = true) →
    apply_late_policy late_days g = g.
Proof.
  (* FILL IN HERE *) Admitted.
```

□

The following theorem states that, if a student has between 9 and 16 late days, their final grade is lowered by one step.

Exercise: 2 stars, standard (graded lowered once)

```
Theorem grade_lowered_once :
  ∀ (late_days : nat) (g : grade),
    (late_days <? 9 = false) →
    (late_days <? 17 = true) →
    (apply_late_policy late_days g) = (lower_grade g).
Proof.
  (* FILL IN HERE *) Admitted.
```

□

End LateDays.

Binary Numerals

Exercise: 3 stars, standard (binary)

We can generalize our unary representation of natural numbers to the more efficient binary representation by treating a binary number as a sequence of constructors B_0 and B_1 (representing 0s and 1s), terminated by a Z . For comparison, in the unary representation, a number is a sequence of S constructors terminated by an 0 .

For example:

decimal	binary	unary
0	Z	0
1	$B_1 Z$	$S\ 0$
2	$B_0 (B_1 Z)$	$S\ (S\ 0)$
3	$B_1 (B_1 Z)$	$S\ (S\ (S\ 0))$
4	$B_0 (B_0 (B_1 Z))$	$S\ (S\ (S\ (S\ 0)))$
5	$B_1 (B_0 (B_1 Z))$	$S\ (S\ (S\ (S\ (S\ 0))))$
6	$B_0 (B_1 (B_1 Z))$	$S\ (S\ (S\ (S\ (S\ (S\ 0)))))$
7	$B_1 (B_1 (B_1 Z))$	$S\ (S\ (S\ (S\ (S\ (S\ (S\ 0)))))$
8	$B_0 (B_0 (B_0 (B_1 Z)))$	$S\ (S\ (S\ (S\ (S\ (S\ (S\ (S\ 0)))))$

Note that the low-order bit is on the left and the high-order bit is on the right -- the opposite of the way binary numbers are usually written. This choice makes them easier to manipulate.

(Comprehension check: What unary numeral does $B_0 Z$ represent?)

```
Inductive bin : Type :=
| Z
| B0 (n : bin)
| B1 (n : bin).
```

Complete the definitions below of an increment function `incr` for binary numbers, and a function `bin_to_nat` to convert binary numbers to unary numbers.

```
Fixpoint incr (m:bin) : bin
(* REPLACE THIS LINE WITH ":= _your_definition_ ." *). Admitted.

Fixpoint bin_to_nat (m:bin) : nat
(* REPLACE THIS LINE WITH ":= _your_definition_ ." *). Admitted.
```

The following "unit tests" of your increment and binary-to-unary functions should pass after you have defined those functions correctly. Of course, unit tests don't fully demonstrate the correctness of your functions! We'll return to that thought at the end of the next chapter.

```
Example test_bin_incr1 : (incr (B1 Z)) = B0 (B1 Z).
(* FILL IN HERE *) Admitted.

Example test_bin_incr2 : (incr (B0 (B1 Z))) = B1 (B1 Z).
(* FILL IN HERE *) Admitted.

Example test_bin_incr3 : (incr (B1 (B1 Z))) = B0 (B0 (B1 Z)).
(* FILL IN HERE *) Admitted.

Example test_bin_incr4 : bin_to_nat (B0 (B1 Z)) = 2.
(* FILL IN HERE *) Admitted.

Example test_bin_incr5 :
  bin_to_nat (incr (B1 Z)) = 1 + bin_to_nat (B1 Z).
(* FILL IN HERE *) Admitted.

Example test_bin_incr6 :
  bin_to_nat (incr (incr (B1 Z))) = 2 + bin_to_nat (B1 Z).
(* FILL IN HERE *) Admitted.
```

```
Example test_bin_incr7 : bin_to_nat (B0 (B0 (B0 (B1 Z)))) = 8.  
(* FILL IN HERE *) Admitted.
```

□

Optional: Testing Your Solutions

Each SF chapter comes with a test file containing scripts that check whether you have solved the required exercises. If you're using SF as part of a course, your instructor will likely be running these test files to autograde your solutions. You can also use these test files, if you like, to make sure you haven't missed anything.

Important: This step is *optional*: if you've completed all the non-optional exercises and Rocq accepts your answers, this already shows that you are in good shape.

The test file for this chapter is `BasicsTest.v`. To run it, make sure you have saved `Basics.v` to disk. Then first run `rocq compile -Q . LF Basics.v` and then run `rocq compile -Q . LF BasicsTest.v`; or, if you have make installed, you can run `make BasicsTest.vo`. (Make sure you do this in a directory that also contains a file named `_CoqProject` containing the single line `-Q . LF`.)

If you accidentally deleted an exercise or changed its name, then `make BasicsTest.vo` will fail with an error that tells you the name of the missing exercise. Otherwise, you will get a lot of useful output:

- First will be all the output produced by `Basics.v` itself.
- Second, for each required exercise, there is a report that tells you its point value (the number of stars or some fraction thereof if there are multiple parts to the exercise), whether its type is ok, and what assumptions it relies upon.

If the *type* is not ok, it means you proved the wrong thing: most likely, you accidentally modified the theorem statement while you were proving it. The autograder won't give you any points in this case, so make sure to correct the theorem.

The *assumptions* are any unproved theorems which your solution relies upon. "Closed under the global context" is a fancy way of saying "none": you have solved the exercise. (Hooray!) On the other hand, a list of axioms means you haven't fully solved the exercise. (But see below regarding "Allowed Axioms.") If the exercise name itself is in the list, that means you haven't solved it; probably you have **Admitted** it.

- Third, you will see the maximum number of points in standard and advanced versions of the assignment. That number is based on the number of stars in the non-optional exercises. (In the present file, there are no advanced exercises.)
- Fourth, you will see a list of "Allowed Axioms". These are unproven theorems that your solution is permitted to depend upon, aside from the fundamental axioms of Rocq's logic. You'll probably see something about `functional_extensionality` for this chapter; we'll cover what that means in a later chapter.
- Finally, you will see a summary of whether you have solved each exercise. Note that summary does not include the critical information of whether the type is ok (that is, whether you accidentally changed the theorem statement): you have to look above for that information.

Exercises that are manually graded will also show up in the output. But since they have to be graded by a human, the test script won't be able to tell you much about them.

Index

This page has been generated by coqdoc